

Python Files And SQLite

Today we move from text inspection to persistent structured data.

Today

- read a CSV with Python
- create a SQLite database
- load ticket records
- query by category
- find high priority open tickets

Why it matters

- apps need stored data
- databases make queries safer
- SQLite requires no server
- Python includes sqlite3
- persistence survives restart

Same Data, More Structure

Bash gives quick answers. Python and SQLite give repeatable application logic.

Bash was good for

- quick search
- line counts
- simple category counts
- ad hoc reports
- working directly in terminal

Python plus SQLite is good for

- structured loading
- clean functions
- database storage
- more precise queries
- building toward an app

Memory, Files, Databases

Different storage choices fit different jobs.

Simple storage

- memory list: easy, disappears
- text file: easy, less structure
- CSV: shareable table format

Database storage

- SQLite: one local file
- PostgreSQL: production server
- MySQL: production server
- DuckDB: analytics-friendly local database

CSV Is Text That Looks Like A Table

Python can read CSV files using a built-in module.

```
id,student_name,category,priority,status
1,Ari,password,normal,open
2,Blake,network,high,open
```

```
import csv
reader = csv.DictReader(file)
```

DictReader uses the header row as field names.

Tables, Rows, Columns, IDs

These words show up in almost every database conversation.

Database words

- table: related records
- row: one record
- column: one field
- primary key: unique ID
- query: request for data

Ticket example

- table: tickets
- row: one helpdesk ticket
- column: priority
- ID: ticket number
- query: open high tickets

SQLite Stores A Database In One File

For beginner apps, this avoids server setup while still teaching database thinking.

tickets.csv

- > Python loader script
- > tickets.db
 - > SQL queries
 - > report output

The database file can be listed with `ls -l`.

Create The Tickets Table

A schema describes the shape of the data.

```
CREATE TABLE IF NOT EXISTS tickets (  
  id INTEGER PRIMARY KEY,  
  student_name TEXT NOT NULL,  
  category TEXT NOT NULL,  
  priority TEXT NOT NULL,  
  status TEXT NOT NULL,  
  issue TEXT NOT NULL  
);
```

NOT NULL means the field is required.

Load CSV Rows Into SQLite

Python reads each CSV row and inserts it into the table.

```
with CSV_PATH.open(newline='') as file:
    reader = csv.DictReader(file)
    for row in reader:
        connection.execute(
            "INSERT INTO tickets VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
            (...)
        )
    connection.commit()
```

Question marks are placeholders for safe values.

Ask The Database Questions

SQL can summarize and filter records.

```
SELECT category, COUNT(*) AS total  
FROM tickets  
GROUP BY category  
ORDER BY total DESC;
```

```
SELECT id, student_name, category, issue  
FROM tickets  
WHERE status = 'open' AND priority = 'high';
```

GROUP BY summarizes. WHERE filters.

The Database Survives Program Restart

This is the difference between practice code and app behavior.

Memory

- fast
- easy for demos
- gone after restart
- not enough for real tickets

SQLite file

- saved on disk
- can be reopened
- can be queried later
- first step toward real app storage

Debug Python And SQLite Calmly

Most problems are path, filename, indentation, or schema issues.

Likely errors

- tickets.csv not found
- tickets.db in wrong folder
- IndentationError
- KeyError from CSV header
- SQL typo
- forgot connection.commit

Debug checks

- pwd
- ls -l
- head tickets.csv
- read traceback file and line
- check column names
- rerun loader

Query Without Writing More Python

If the sqlite3 command exists, students can inspect the database directly.

```
$ sqlite3 tickets.db
sqlite> .tables
sqlite> SELECT COUNT(*) FROM tickets;
sqlite> SELECT category, COUNT(*) FROM tickets GROUP BY category;
sqlite> .quit
```

If sqlite3 is missing, Python query output is enough.

This Is The Data Layer Of Apps

Almost every useful application needs some version of this idea.

Examples

- helpdesk systems
- inventory apps
- dashboards
- QA tracking
- security findings
- AI tool logs

Transferable concepts

- records
- IDs
- schema
- queries
- persistence
- error handling

Brightspace Submission

Students submit scripts, output, and a short explanation of persistence.

Evidence

- python version
- ls -l showing tickets.db
- load_tickets.py
- analyze_tickets.py
- query output

Writing

- what file stores the database
- most common category
- open high tickets
- CSV vs database
- why apps need persistence

What Success Looks Like Today

Students can move data from a file into a persistent database.

Students should be able to say

- I can read a CSV with Python
- I can create a SQLite database
- I can load rows into a table
- I can run a query
- I can explain persistence

Next class

- FastAPI
- HTTP GET and POST
- JSON
- automatic docs
- debugging an app